

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

Lucas Gomes da Cunha

ESTUDO DE CASO UTILIZANDO O MONGODB

CAMPOS DO JORDÃO

2026

LUCAS GOMES DA CUNHA

Estudo de caso utilizando o MongoDB: Sistema de Gestão Cartório

Projeto final NoSQL da matéria Banco de Dados II em Análise e Desenvolvimento de Sistemas pelo Instituto Federal de São Paulo - IFSP.

Orientador: Prof. Paulo Giovani de Faria Zeferino

CAMPOS DO JORDÃO

2026

RESUMO

Este trabalho apresenta a adaptação do banco de dados relacional do Sistema de Gestão Cartorária para um banco de dados não relacional (NoSQL), utilizando o Sistema Gerenciador de Banco de Dados MongoDB. O objetivo consiste em demonstrar a aplicação dos conceitos de bancos de dados NoSQL por meio da conversão da estrutura relacional desenvolvida anteriormente para um modelo orientado a documentos, preservando as regras de negócio e explorando as características de flexibilidade e escalabilidade oferecidas pelo MongoDB. A metodologia adotada baseou-se na análise do projeto relacional existente, no estudo dos principais modelos de bancos de dados NoSQL e na implementação da nova estrutura em coleções e documentos. O embasamento teórico fundamenta-se nas obras de Peter Rob e Carlos Coronel, Pramod J. Sadalage e Martin Fowler, e Kyle Banker, que abordam os conceitos de modelagem de bancos de dados, tecnologias NoSQL e a utilização do MongoDB. Para validar a estrutura desenvolvida, foram utilizadas as mesmas informações do projeto relacional, realizando-se a inserção dos dados nas coleções e a elaboração de trinta consultas representativas das operações do sistema cartorário. Os resultados obtidos demonstram que a modelagem orientada a documentos atendeu aos requisitos do sistema, permitindo o armazenamento e a recuperação eficiente das informações, além de evidenciar as diferenças e as vantagens da abordagem NoSQL em relação ao modelo relacional.

Palavras-chave: Banco de Dados NoSQL; MongoDB; Modelagem Orientada a Documentos; Sistema Cartorário; SQL Server.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Resultado Consulta 1 (MongoDB)	22
FIGURA 2 – Resultado Consulta 2 (MongoDB)	23
FIGURA 3 – Resultado Consulta 3 (MongoDB)	23
FIGURA 4 – Resultado Consulta 4 (MongoDB)	24
FIGURA 5 – Resultado Consulta 5 (MongoDB)	25
FIGURA 6 – Resultado Consulta 6 (MongoDB)	25
FIGURA 7 – Resultado Consulta 7 (MongoDB)	26
FIGURA 8 – Resultado Consulta 8 (MongoDB)	27
FIGURA 9 – Resultado Consulta 9 (MongoDB)	27
FIGURA 10 – Resultado Consulta 10 (MongoDB)	28
FIGURA 11 – Resultado Consulta 11 (MongoDB)	29
FIGURA 12 – Resultado Consulta 12 (MongoDB)	30
FIGURA 13 – Resultado Consulta 13 (MongoDB)	30
FIGURA 14 – Resultado Consulta 14 (MongoDB)	31
FIGURA 15 – Resultado Consulta 15 (MongoDB)	32
FIGURA 16 – Resultado Consulta 16 (MongoDB)	32
FIGURA 17 – Resultado Consulta 17 (MongoDB)	33
FIGURA 18 – Resultado Consulta 18 (MongoDB)	34
FIGURA 19 – Resultado Consulta 19 (MongoDB)	34
FIGURA 20 – Resultado Consulta 20 (MongoDB)	35
FIGURA 21 – Resultado Consulta 21 (MongoDB)	36
FIGURA 22 – Resultado Consulta 22 (MongoDB)	37
FIGURA 23 – Resultado Consulta 23 (MongoDB)	37

FIGURA 24 – Resultado Consulta 24 (MongoDB)	38
FIGURA 25 – Resultado Consulta 25 (MongoDB)	39
FIGURA 26 – Resultado Consulta 26 (MongoDB)	39
FIGURA 27 – Resultado Consulta 27 (MongoDB)	40
FIGURA 28 – Resultado Consulta 28 (MongoDB)	41
FIGURA 29 – Resultado Consulta 29 (MongoDB)	42
FIGURA 30 – Resultado Consulta 30 (MongoDB)	42

LISTA DE ALGORITMOS

ALGORITMO 1 – Inserção dos Documentos – Coleção Clientes (MongoDB)	13
ALGORITMO 2 – Exemplo de Documento (MongoDB)	13
ALGORITMO 3 – Inserção dos Documentos – Clientes (MongoDB)	17
ALGORITMO 4 – Inserção dos Documentos – Funcionários (MongoDB)	18
ALGORITMO 5 – Inserção dos Documentos – Atendimentos (MongoDB)	18
ALGORITMO 6 – Inserção dos Documentos – Atendimentos (MongoDB) Parte 1	19
ALGORITMO 7 – Inserção dos Documentos – Atendimentos (MongoDB) Parte 2	20
ALGORITMO 8 – Consulta 1 (MongoDB)	22
ALGORITMO 9 – Consulta 2 (MongoDB)	22
ALGORITMO 10 – Consulta 3 (MongoDB)	23
ALGORITMO 11 – Consulta 4 (MongoDB)	24
ALGORITMO 12 – Consulta 5 (MongoDB)	24
ALGORITMO 13 – Consulta 6 (MongoDB)	25
ALGORITMO 14 – Consulta 7 (MongoDB)	26
ALGORITMO 15 – Consulta 8 (MongoDB)	26
ALGORITMO 16 – Consulta 9 (MongoDB)	27
ALGORITMO 17 – Consulta 10 (MongoDB)	28
ALGORITMO 18 – Consulta 11 (MongoDB)	29
ALGORITMO 19 – Consulta 12 (MongoDB)	29
ALGORITMO 20 – Consulta 13 (MongoDB)	30
ALGORITMO 21 – Consulta 14 (MongoDB)	31
ALGORITMO 22 – Consulta 15 (MongoDB)	31

ALGORITMO 23 – Consulta 16 (MongoDB)	32
ALGORITMO 24 – Consulta 17 (MongoDB)	33
ALGORITMO 25 – Consulta 18 (MongoDB)	33
ALGORITMO 26 – Consulta 19 (MongoDB)	34
ALGORITMO 27 – Consulta 20 (MongoDB)	35
ALGORITMO 28 – Consulta 21 (MongoDB)	36
ALGORITMO 29 – Consulta 22 (MongoDB)	36
ALGORITMO 30 – Consulta 23 (MongoDB)	37
ALGORITMO 31 – Consulta 24 (MongoDB)	38
ALGORITMO 32 – Consulta 25 (MongoDB)	38
ALGORITMO 33 – Consulta 26 (MongoDB)	39
ALGORITMO 34 – Consulta 27 (MongoDB)	40
ALGORITMO 35 – Consulta 28 (MongoDB)	41
ALGORITMO 36 – Consulta 29 (MongoDB)	41
ALGORITMO 37 – Consulta 30 (MongoDB)	42

SUMÁRIO

1	INTRODUÇÃO _____	08
1.1	Objetivos _____	08
1.2	Justificativa _____	08
1.3	Aspectos Metodológicos _____	09
1.4	Aporte Teórico _____	10
2	METODOLOGIA E FUNDAMENTAÇÃO TEÓRICA _____	11
2.1	Bancos de Dados Não Relacionais _____	11
2.2	Modelos de Dados no NoSQL _____	12
2.3	SGBD MongoDB e o Modelo Orientado a Documentos _____	14
2.4	Trabalhos Relacionados _____	16
3	SISTEMA DE GESTÃO DE CARTÓRIO em NoSQL UTILIZANDO MONGODB _____	17
3.1	Criação de Coleções e Inserção de Dados (MongoDB) _____	17
3.2	Consultas Propostas _____	21
4	CONCLUSÃO _____	43

1 INTRODUÇÃO

1.1 Objetivos

O presente trabalho tem como objetivo geral apresentar os bancos de dados não relacionais e desenvolver um projeto prático utilizando o MongoDB, baseado no Sistema de Gestão Cartorária elaborado durante o Projeto de Banco de Dados Relacional.

Para alcançar esse objetivo geral, foram definidos os seguintes objetivos específicos:

- Descrever o conceito de bancos de dados não relacionais e suas características fundamentais;
- Apresentar os principais modelos de dados existentes na categoria NoSQL, como orientado a documentos, chave-valor, colunar e baseado em grafos;
- Contextualizar o MongoDB como SGBD, descrevendo seu modelo de dados e suas principais características;
- Adaptar o projeto do Sistema de Gestão Cartorária para o paradigma não relacional, utilizando o MongoDB como sistema gerenciador;
- Demonstrar a criação de coleções, inserção de documentos e execução de consultas no MongoDB, evidenciando as diferenças em relação ao modelo relacional.

1.2 Justificativa

Este trabalho se justifica pela crescente adoção de bancos de dados NoSQL em aplicações que demandam flexibilidade, escalabilidade horizontal e alta disponibilidade, características difíceis de alcançar com eficiência nos sistemas relacionais tradicionais. Compreender esse paradigma é essencial para um

profissional de Análise e Desenvolvimento de Sistemas, pois o mercado de tecnologia exige o domínio de diferentes abordagens de persistência de dados.

Além disso, ao utilizar como base o Sistema de Gestão Cartorária já desenvolvido, é possível estabelecer uma comparação direta entre as duas abordagens, evidenciando na prática as transformações estruturais necessárias ao migrar de um modelo relacional normalizado para um modelo orientado a documentos, onde o conceito de *embedding* e referências entre documentos substitui as chaves estrangeiras e os relacionamentos das tabelas.

1.3 Aspectos Metodológicos

O presente trabalho foi desenvolvido com base em pesquisa bibliográfica e prática, utilizando conceitos relacionados a bancos de dados não relacionais e ao sistema gerenciador MongoDB. Inicialmente, foi realizado o levantamento teórico sobre o paradigma NoSQL, seus modelos de dados e suas principais diferenças em relação ao modelo relacional.

Em seguida, foi utilizado o Sistema de Gestão Cartorária desenvolvido previamente no Microsoft SQL Server como referência para a adaptação ao ambiente não relacional. O banco de dados cartorário, composto pelas entidades Cliente, Funcionário, Documento, Atendimento e Registro, foi reinterpretado no modelo orientado a documentos do MongoDB, onde os dados são organizados em coleções e armazenados no formato BSON (*Binary JSON*).

Por fim, foram desenvolvidos exemplos práticos de criação de coleções, inserção de documentos e execução de consultas no MongoDB, permitindo validar o funcionamento do sistema e comparar os resultados com as consultas SQL realizadas no projeto relacional.

1.4 Aporte Teórico

O desenvolvimento deste trabalho baseia-se nos conceitos de bancos de dados não relacionais apresentados por autores como Pramod J. Sadalage e Martin Fowler, cujos estudos sobre NoSQL oferecem uma visão abrangente sobre os diferentes modelos de dados e seus casos de uso. Complementarmente, a documentação oficial do MongoDB serve como referência técnica para a compreensão do modelo orientado a documentos e das operações disponíveis no sistema.

O trabalho também se fundamenta nos conceitos de modelagem de dados relacionais estudados por Peter Rob e Carlos Coronel, utilizados como ponto de partida para a comparação entre os dois paradigmas. A experiência prática adquirida no desenvolvimento do projeto relacional anterior fornece a base necessária para compreender as adaptações estruturais requeridas pela migração para o ambiente NoSQL, enriquecendo a análise e tornando o estudo mais contextualizado e aplicado.

2 METODOLOGIA E FUNDAMENTAÇÃO TEÓRICA

A fundamentação teórica deste trabalho baseia-se nos conceitos de bancos de dados não relacionais, seus modelos de armazenamento e nas características do MongoDB como sistema gerenciador. Segundo Sadalage e Fowler (2013), os bancos de dados NoSQL surgiram como resposta às limitações dos sistemas relacionais diante do crescimento massivo de dados e da necessidade de escalabilidade em ambientes distribuídos.

2.1 Bancos de Dados Não Relacionais

Os bancos de dados não relacionais, conhecidos como NoSQL (*Not Only SQL*), são sistemas de gerenciamento de dados que não utilizam o modelo tabular tradicional baseado em linhas e colunas como estrutura principal de armazenamento. Ao contrário dos bancos de dados relacionais, que organizam as informações em tabelas com esquemas rígidos e predefinidos, os bancos NoSQL oferecem esquemas flexíveis ou inexistentes, permitindo que os dados sejam armazenados de formas variadas de acordo com a necessidade da aplicação.

Entre as principais características dos bancos de dados NoSQL, destacam-se:

- **Escalabilidade horizontal:** capacidade de distribuir os dados entre múltiplos servidores, aumentando a capacidade do sistema pela adição de novas máquinas, em vez de aumentar o poder de processamento de um único servidor;
- **Flexibilidade de esquema:** os dados podem ser inseridos sem a necessidade de uma estrutura previamente definida, permitindo que diferentes registros de uma mesma coleção possuam atributos distintos;
- **Alta disponibilidade:** os sistemas NoSQL são projetados para operar de forma contínua, mesmo diante de falhas em parte da infraestrutura, por meio de replicação e redundância de dados;

- **Desempenho elevado:** ao eliminar a necessidade de operações complexas de junção entre tabelas (*joins*), os bancos NoSQL conseguem recuperar grandes volumes de dados com maior velocidade em determinados cenários;
- **Suporte a dados não estruturados e semiestruturados:** diferentemente dos bancos relacionais, que exigem dados bem estruturados, os sistemas NoSQL são capazes de armazenar textos livres, arquivos JSON, imagens e outros formatos variados.

É importante ressaltar que os bancos de dados NoSQL não substituem os sistemas relacionais em todos os cenários. Cada abordagem apresenta vantagens e limitações, e a escolha entre elas deve considerar os requisitos específicos do sistema, como o volume de dados, a necessidade de consistência, o tipo de consultas realizadas e a estrutura das informações.

2.2 Modelos de Dados no NoSQL

Os bancos de dados não relacionais são classificados em diferentes categorias de acordo com o modelo de dados utilizado para organizar e armazenar as informações. Cada modelo apresenta características próprias e é mais adequado para determinados tipos de aplicação. As quatro principais categorias são: orientado a documentos, chave-valor, colunar e baseado em grafos.

- **Modelo Orientado a Documentos:** O modelo orientado a documentos armazena as informações em documentos semiestruturados, geralmente no formato JSON (*JavaScript Object Notation*) ou BSON (*Binary JSON*). Cada documento é uma unidade independente de dados que pode conter campos, valores, arrays e até documentos aninhados, eliminando a necessidade de distribuir as informações entre múltiplas tabelas relacionadas. Nesse modelo, os documentos são agrupados em coleções, que funcionam de forma análoga às tabelas dos bancos relacionais, porém sem a obrigatoriedade de um esquema fixo. Isso significa que dois documentos de uma mesma coleção podem ter estruturas completamente diferentes. Os principais representantes

desse modelo são o **MongoDB** e o **CouchDB**. Esse modelo é especialmente indicado para aplicações que lidam com dados hierárquicos ou semiestruturados, como catálogos de produtos, perfis de usuários e sistemas de gerenciamento de conteúdo.

```

1 {
2   "id_cliente": 1,
3   "nome": "João Silva",
4   "cpf": "12345678901",
5   "contato": {
6     "telefone": "(11)99999-1111",
7     "email": "joao@gmail.com"
8   },
9   "documentos": [
10    {
11      "tipo": "RG",
12      "numero": "MG123456",
13      "orgao_emissor": "SSP"
14    }
15  ]
16 }
```

Algoritmo 1 – Inserção dos Documentos – Coleção Clientes (MongoDB)

- **Modelo Chave-Valor:** O modelo chave-valor é o mais simples entre os paradigmas NoSQL. Nele, cada registro é armazenado como um par composto por uma chave única e um valor associado, que pode ser uma string, um número, um objeto binário ou qualquer outro tipo de dado. O acesso aos dados é feito exclusivamente pela chave, tornando as operações de leitura e escrita extremamente rápidas. Por sua simplicidade e alto desempenho, esse modelo é amplamente utilizado em cenários que exigem respostas em tempo real, como sistemas de cache, gerenciamento de sessões de usuários e filas de mensagens. Os principais representantes são o **Redis** e o **Amazon DynamoDB**.

```

1 "cliente:001" → { nome: "João Silva", cpf: "12345678901" }
2 "sessao:abc123" → { usuario: "joao", expiracao: "2026-06-30" }
```

Algoritmo 2 – Exemplo de Documento (MongoDB)

- **Modelo Colunar:** O modelo colunar, também chamado de modelo orientado a colunas ou wide-column, organiza os dados em famílias de colunas em vez de linhas. Diferentemente dos bancos relacionais, onde todos os valores de uma linha são armazenados juntos, no modelo colunar os valores de cada

coluna são agrupados e armazenados de forma independente. Isso torna as operações de leitura e agregação sobre colunas específicas muito mais eficientes, pois apenas as colunas necessárias precisam ser acessadas. Esse modelo é especialmente vantajoso para aplicações de análise de grandes volumes de dados (*Big Data*), relatórios e sistemas de inteligência de negócios. Os principais representantes são o **Apache Cassandra** e o **HBase**

RowKey	nome	cpf	telefone
001	João Silva	12345678901	(11)99999-1111
002	Maria Oliveira	12345678902	(11)99999-2222

- **Modelo Baseado em Grafos:** O modelo baseado em grafos utiliza estruturas matemáticas de grafos para representar e armazenar os dados. Nesse modelo, as informações são organizadas em nós (*nodes*), que representam as entidades, arestas (*edges*), que representam os relacionamentos entre as entidades, e propriedades, que descrevem os atributos de nós e arestas. Essa estrutura torna o modelo de grafos extremamente eficiente para consultas que envolvem relacionamentos complexos e profundos entre os dados. Aplicações como redes sociais, sistemas de recomendação, detecção de fraudes e análise de rotas são casos de uso típicos para esse modelo. Os principais representantes são o **Neo4j** e o **Amazon Neptune**.

(João Silva) —[REALIZOU]—> (Atendimento: Registro de Casamento)
 (João Silva) —[POSSUI]—> (Documento: RG MG123456)
 (Paulo Henrique) —[RESPONSÁVEL]—> (Atendimento: Registro de Casamento)

2.3 SGBD MongoDB e o Modelo Orientado a Documentos

O MongoDB é um sistema gerenciador de banco de dados não relacional de código aberto, classificado dentro da categoria de bancos de dados orientados a

documentos. Desenvolvido pela MongoDB Inc. e lançado em 2009, o sistema foi projetado para oferecer alta performance, escalabilidade horizontal e flexibilidade na organização dos dados, características que o tornaram uma das soluções NoSQL mais adotadas no mercado mundial. De acordo com Banker (2013), o MongoDB combina a simplicidade do modelo de documentos com recursos sofisticados de indexação e consulta, aproximando-se, em termos de expressividade, dos sistemas relacionais tradicionais.

Estruturalmente, o MongoDB organiza os dados de maneira distinta dos bancos relacionais. Enquanto no modelo relacional as informações são distribuídas em tabelas compostas por linhas e colunas com esquema rígido e predefinido, no MongoDB os dados são agrupados em coleções, que funcionam como contêineres de documentos. Cada documento é uma unidade autossuficiente de informação, representada internamente no formato BSON — Binary JSON —, uma versão binária e otimizada do formato JSON, capaz de suportar tipos de dados ricos como datas, números de precisão decimal e objetos aninhados. Diferentemente das linhas de uma tabela relacional, dois documentos de uma mesma coleção podem possuir estruturas completamente diferentes entre si, sem que isso represente um problema para o sistema.

No âmbito do Sistema de Gestão Cartorária, essa característica do MongoDB traz vantagens práticas significativas. No modelo relacional desenvolvido anteriormente, as informações de um atendimento cartorário estavam distribuídas entre três tabelas distintas — Atendimento, Registro e Cliente —, exigindo operações de junção para que todos os dados fossem recuperados de forma consolidada. No modelo orientado a documentos, essa fragmentação deixa de existir: o registro cartorário vinculado a um atendimento, contendo livro, folha e termo, passa a ser armazenado como um subdocumento aninhado dentro do próprio documento do atendimento. De forma análoga, os documentos de identificação de cada cliente — RG, CPF, certidões — são incorporados como uma lista embutida no documento do cliente, eliminando a necessidade da tabela Documento como entidade separada.

Essa estratégia de agrupamento, denominada *embedding*, reduz o número de operações de leitura necessárias para recuperar informações relacionadas, uma vez

que todos os dados de um atendimento ou de um cliente passam a residir em um único documento, acessível por meio de uma consulta direta à coleção correspondente.

2.4 Trabalhos Relacionados

Diversos sistemas e estudos acadêmicos exploram a aplicação de bancos de dados NoSQL em ambientes administrativos e de gestão documental. A flexibilidade do modelo orientado a documentos tem se mostrado vantajosa em sistemas que lidam com informações heterogêneas e que precisam evoluir sem grandes impactos estruturais, como plataformas de gestão de conteúdo, prontuários eletrônicos e sistemas de controle documental.

No contexto do presente trabalho, a experiência adquirida no desenvolvimento do Sistema de Gestão Cartorária com o Microsoft SQL Server serve como base comparativa para a implementação no MongoDB. As obras de Sadalage e Fowler sobre NoSQL e a documentação oficial do MongoDB fornecem o suporte técnico e conceitual necessário para a adaptação do modelo relacional ao paradigma orientado a documentos, evidenciando tanto as vantagens quanto os desafios dessa transição.

3 SISTEMA DE GESTÃO DE CARTÓRIO EM NoSQL UTILIZANDO O MONGODB

3.1 Criação de Coleções e Inserção de Dados (MongoDB)

Na etapa anterior da disciplina, foi realizado o projeto do banco de dados relacional do Sistema de Gestão Cartorária, estruturado por meio dos níveis de modelagem conceitual, lógica e física. Esse processo possibilitou o levantamento e a organização dos requisitos de negócio do cartório, resultando na definição das entidades Cliente, Funcionário, Documento, Atendimento e Registro, bem como de seus respectivos relacionamentos.

Para a adaptação do modelo relacional para o modelo não relacional, as tabelas do banco de dados relacional foram convertidas em coleções, enquanto os registros passaram a ser armazenados como documentos. Foram criadas três coleções principais: clientes (com documentos de identificação aninhados), funcionarios e atendimentos (com registros cartorários aninhados). Após a criação das coleções, foram inseridos dados representativos do sistema cartorário, correspondentes aos mesmos registros utilizados no projeto relacional

Algoritmo 3 – Inserção dos Documentos – Clientes (MongoDB)

```

1 db.clientes.insertMany([
2   { _id: 1, nome: "João Silva", cpf: "12345678901", rg: "MG123456",
3     telefone: "(11)99999-1111", email: "joao@gmail.com",
4     endereco: "Rua das Flores, 100",
5     documentos: [{ id_documento: 1, tipo_documento: "RG",
6       numero_documento: "MG123456", data_emissao: new Date("2018-05-10"),
7       orgao_emissor: "SSP" }] },
8   { _id: 2, nome: "Maria Oliveira", cpf: "12345678902", rg: "SP987654",
9     telefone: "(11)99999-2222", email: "maria@gmail.com",
10    endereco: "Av. Central, 200",
11    documentos: [{ id_documento: 2, tipo_documento: "CPF",
12      numero_documento: "12345678902", data_emissao: new
13      Date("2017-08-15"),
14      orgao_emissor: "Receita Federal" }] },
15   { _id: 3, nome: "Carlos Souza", cpf: "12345678903", rg: "RJ654321",
16     telefone: "(11)99999-3333", email: "carlos@gmail.com",
17     endereco: "Rua Azul, 45",

```

```

17     documentos: [{ id_documento: 3, tipo_documento: "Certidão de
    Nascimento",
18         numero_documento: "CN1001", data_emissao: new Date("2001-03-20"),
19         orgao_emissor: "Cartório Central" }] },
20     // ... demais 12 clientes seguindo o mesmo padrão
21 ])
```

Algoritmo 4 – Inserção dos Documentos – Funcionários (MongoDB)

```

1 db.funcionarios.insertMany([
2   { _id: 1, nome: "Paulo Henrique", cargo: "Escreveinte",
3     cpf: "98765432101", telefone: "(11)97777-1111",
4     salario: 3500.00, data_admissao: new Date("2022-01-10") },
5   { _id: 2, nome: "Roberta Lima", cargo: "Tabeliã",
6     cpf: "98765432102", telefone: "(11)97777-2222",
7     salario: 7000.00, data_admissao: new Date("2020-03-15") },
8   { _id: 3, nome: "Felipe Souza", cargo: "Atendente",
9     cpf: "98765432103", telefone: "(11)97777-3333",
10    salario: 2800.00, data_admissao: new Date("2021-07-20") },
11   { _id: 4, nome: "Amanda Costa", cargo: "Escreveinte",
12     cpf: "98765432104", telefone: "(11)97777-4444",
13     salario: 3600.00, data_admissao: new Date("2023-02-01") },
14   { _id: 5, nome: "Renato Alves", cargo: "Supervisor",
15     cpf: "98765432105", telefone: "(11)97777-5555",
16     salario: 5000.00, data_admissao: new Date("2019-09-12") }
17 ])
```

Algoritmo 5 – Inserção dos Documentos – Atendimentos (MongoDB)

```

1 db.atendimentos.insertMany([
2   { _id: 1, id_cliente: 1, id_funcionario: 1,
3     data_atendimento: new Date("2026-01-10T09:00:00"),
4     tipo_servico: "Reconhecimento de Firma",
5     valor: 25.00, status_atendimento: "Concluído",
6     registro: { tipo_registro: "Firma", livro: "L01",
7       folha: "F12", termo: "T001",
8       data_registro: new Date("2026-01-10") } },
9   { _id: 2, id_cliente: 2, id_funcionario: 2,
10     data_atendimento: new Date("2026-01-11T10:30:00"),
11     tipo_servico: "Autenticação de Documento",
12     valor: 15.00, status_atendimento: "Concluído",
13     registro: { tipo_registro: "Documento", livro: "L01",
14       folha: "F13", termo: "T002",
15       data_registro: new Date("2026-01-11") } },
16   // ... demais 13 atendimentos seguindo o mesmo padrão
17 ])
```

Algoritmo 6 – Inserção dos Documentos – Atendimentos (MongoDB) Parte 1

```
1 db.atendimentos.insertMany([
2   { _id: 1, id_cliente: 1, id_funcionario: 1,
3     data_atendimento: new Date("2026-01-10T09:00:00"),
4     tipo_servico: "Reconhecimento de Firma",
5     valor: 25.00, status_atendimento: "Concluído",
6     registro: { tipo_registro: "Firma", livro: "L01",
7     folha: "F12", termo: "T001",
8     data_registro: new Date("2026-01-10") } },
9   { _id: 2, id_cliente: 2, id_funcionario: 2,
10     data_atendimento: new Date("2026-01-11T10:30:00"),
11     tipo_servico: "Autenticação de Documento",
12     valor: 15.00, status_atendimento: "Concluído",
13     registro: { tipo_registro: "Documento", livro: "L01",
14     folha: "F13", termo: "T002",
15     data_registro: new Date("2026-01-11") } },
16   { _id: 3, id_cliente: 3, id_funcionario: 3,
17     data_atendimento: new Date("2026-01-12T11:15:00"),
18     tipo_servico: "Emissão de Certidão",
19     valor: 40.00, status_atendimento: "Concluído",
20     registro: { tipo_registro: "Certidão", livro: "L02",
21     folha: "F20", termo: "T003",
22     data_registro: new Date("2026-01-12") } },
23   { _id: 4, id_cliente: 4, id_funcionario: 4,
24     data_atendimento: new Date("2026-01-13T14:00:00"),
25     tipo_servico: "Registro de Nascimento",
26     valor: 120.00, status_atendimento: "Concluído",
27     registro: { tipo_registro: "Nascimento", livro: "L03",
28     folha: "F01", termo: "T004",
29     data_registro: new Date("2026-01-13") } },
30   { _id: 5, id_cliente: 5, id_funcionario: 5,
31     data_atendimento: new Date("2026-01-14T15:45:00"),
32     tipo_servico: "Registro de Casamento",
33     valor: 250.00, status_atendimento: "Pendente",
34     registro: { tipo_registro: "Casamento", livro: "L03",
35     folha: "F02", termo: "T005",
36     data_registro: new Date("2026-01-14") } },
37   { _id: 6, id_cliente: 6, id_funcionario: 1,
38     data_atendimento: new Date("2026-01-15T08:20:00"),
39     tipo_servico: "Reconhecimento de Firma",
40     valor: 25.00, status_atendimento: "Concluído",
41     registro: { tipo_registro: "Firma", livro: "L01",
42     folha: "F14", termo: "T006",
43     data_registro: new Date("2026-01-15") } },
44   { _id: 7, id_cliente: 7, id_funcionario: 2,
45     data_atendimento: new Date("2026-01-16T13:00:00"),
```

```

46     tipo_servico: "Autenticação de Documento",
47     valor: 15.00, status_atendimento: "Concluído",
48     registro: { tipo_registro: "Documento", livro: "L01",
49     folha: "F15", termo: "T007",
50     data_registro: new Date("2026-01-16") } },
51   { _id: 8, id_cliente: 8, id_funcionario: 3,
52     data_atendimento: new Date("2026-01-17T16:10:00"),
53     tipo_servico: "Emissão de Certidão",
54     valor: 40.00, status_atendimento: "Concluído",
55     registro: { tipo_registro: "Certidão", livro: "L02",
56     folha: "F21", termo: "T008",
57     data_registro: new Date("2026-01-17") } },
58 ]))

```

Algoritmo 7 – Inserção dos Documentos – Atendimentos (MongoDB) Parte 2

```

1 db.atendimentos.insertMany([
2   { _id: 9, id_cliente: 9, id_funcionario: 4,
3     data_atendimento: new Date("2026-01-18T09:50:00"),
4     tipo_servico: "Registro de Imóvel",
5     valor: 500.00, status_atendimento: "Pendente",
6     registro: { tipo_registro: "Imóvel", livro: "L04",
7     folha: "F10", termo: "T009",
8     data_registro: new Date("2026-01-18") } },
9   { _id: 10, id_cliente: 10, id_funcionario: 5,
10     data_atendimento: new Date("2026-01-19T10:00:00"),
11     tipo_servico: "Reconhecimento de Firma",
12     valor: 25.00, status_atendimento: "Concluído",
13     registro: { tipo_registro: "Firma", livro: "L01",
14     folha: "F16", termo: "T010",
15     data_registro: new Date("2026-01-19") } },
16   { _id: 11, id_cliente: 11, id_funcionario: 1,
17     data_atendimento: new Date("2026-01-20T11:30:00"),
18     tipo_servico: "Autenticação de Documento",
19     valor: 15.00, status_atendimento: "Concluído",
20     registro: { tipo_registro: "Documento", livro: "L01",
21     folha: "F17", termo: "T011",
22     data_registro: new Date("2026-01-20") } },
23   { _id: 12, id_cliente: 12, id_funcionario: 2,
24     data_atendimento: new Date("2026-01-21T14:40:00"),
25     tipo_servico: "Registro de Nascimento",
26     valor: 120.00, status_atendimento: "Concluído",
27     registro: { tipo_registro: "Nascimento", livro: "L03",

```

3.2 Consultas Propostas

Após a modelagem das coleções e a inserção dos registros no MongoDB, deu-se início à etapa de validação da base de dados, composta pela elaboração e execução de um conjunto de trinta consultas desenvolvidas por meio da linguagem de consulta nativa do SGBD. Essa etapa tem por finalidade comprovar que a estrutura implementada é capaz de atender às demandas operacionais do sistema cartorário, permitindo a recuperação eficiente e precisa das informações armazenadas nas coleções clientes, funcionarios e atendimentos.

Para contemplar a diversidade de operações disponíveis no MongoDB, as consultas foram elaboradas de forma a abranger diferentes recursos do sistema, como filtragem por campos simples e aninhados, projeção seletiva de atributos, ordenação de resultados, agrupamento e sumarização de dados por meio do pipeline de agregação, cálculo de métricas como soma, média e contagem, junção entre coleções por meio do operador \$lookup, aplicação de expressões condicionais e manipulação de campos de data. Dessa maneira, o conjunto de consultas reflete situações reais de uso do sistema, como a geração de relatórios gerenciais, a identificação de atendimentos pendentes, o cálculo de receita por tipo de serviço e a localização de clientes por critérios específicos.

Cada consulta é apresentada com sua respectiva descrição, o código desenvolvido na linguagem do MongoDB e o resultado obtido após a execução, possibilitando a análise e a comparação dos dados recuperados em relação às informações inseridas na base de dados.

- **Consulta 1** - Listar clientes e seus atendimentos com valores cobrados

Realiza uma junção entre as coleções clientes e atendimentos, exibindo o nome do cliente, o serviço solicitado e o valor cobrado em cada atendimento.

```
1 db.atendimentos.aggregate([  
2   { $lookup: { from: "clientes", localField: "id_cliente",
```

```

3   foreignField: "_id", as: "cliente" } },
4   { $unwind: "$cliente" },
5   { $project: { _id: 0, Nome_Cliente: "$cliente.nome",
6     Servico_Solicitado: "$tipo_servico",
7     Valor_Cobrado: "$valor" } }
8 ])
```

Algoritmo 8 – Consulta 1 (MongoDB)

```

{ Nome_Cliente: "João Silva",      Servico_Solicitado: "Reconhecimento de Firma",
  Valor_Cobrado: 25 }
{ Nome_Cliente: "Maria Oliveira",  Servico_Solicitado: "Autenticação de
  Documento", Valor_Cobrado: 15 }
{ Nome_Cliente: "Carlos Souza",    Servico_Solicitado: "Emissão de Certidão",
  Valor_Cobrado: 40 }
```

Figura 1 – Resultado Consulta 1 (MongoDB)

- **Consulta 2** - Total de atendimentos realizados por funcionário

Agrupar os atendimentos por funcionário responsável, contando o total de atendimentos realizados por cada colaborador e ordenando o resultado alfabeticamente.

```

1 db.atendimentos.aggregate([
2   { $lookup: { from: "funcionarios", localField: "id_funcionario",
3     foreignField: "_id", as: "funcionario" } },
4   { $unwind: "$funcionario" },
5   { $group: { _id: "$funcionario.nome",
6     Total_Atendimentos: { $sum: 1 } } },
7   { $project: { _id: 0,
8     Funcionario_Responsavel: "$_id",
9     Total_Atendimentos: 1 } },
10  { $sort: { Funcionario_Responsavel: 1 } }
11 ])
```

Algoritmo 9 – Consulta 2 (MongoDB)

```

{ Funcionario_Responsavel: "Amanda Costa",  Total_Atendimentos: 3 }
{ Funcionario_Responsavel: "Felipe Souza",   Total_Atendimentos: 3 }
```

```
{ Funcionario_Responsavel: "Paulo Henrique", Total_Atendimentos: 3 }
{ Funcionario_Responsavel: "Renato Alves",    Total_Atendimentos: 3 }
{ Funcionario_Responsavel: "Roberta Lima",    Total_Atendimentos: 3 }
```

Figura 2 – Resultado Consulta 2 (MongoDB)

- **Consulta 3** - Atendimentos com status Pendente e data agendada

Filtra os atendimentos com status igual a 'Pendente', exibindo o nome do cliente, o serviço solicitado e a data do atendimento agendado.

```
1 db.atendimentos.aggregate([
2   { $match: { status_atendimento: "Pendente" } },
3   { $lookup: { from: "clientes", localField: "id_cliente",
4     foreignField: "_id", as: "cliente" } },
5   { $unwind: "$cliente" },
6   { $project: { _id: 0,
7     Cliente_Pendente: "$cliente.nome",
8     Servico: "$tipo_servico",
9     Data_Agendada: "$data_atendimento" } }
10 ])
```

Algoritmo 10 – Consulta 3 (MongoDB)

```
{ Cliente_Pendente: "Lucas Pereira", Servico: "Registro de Casamento",
Data_Agendada: 2026-01-14 }
{ Cliente_Pendente: "Juliana Rocha", Servico: "Registro de Imóvel",
Data_Agendada: 2026-01-18 }
{ Cliente_Pendente: "Mariana Lopes", Servico: "Registro de Casamento",
Data_Agendada: 2026-01-22 }
```

Figura 3 – Resultado Consulta 3 (MongoDB)

- **Consulta 4** - Receita total gerada por categoria de serviço

Agrupa os atendimentos por tipo de serviço, calculando a receita total gerada por cada categoria e ordenando o resultado alfabeticamente.

```

1 db.atendimentos.aggregate([
2   { $group: { _id: "$tipo_servico",
3     Receita_Total_Gerada: { $sum: "$valor" } } },
4   { $project: { _id: 0,
5     Categoria_Servico: "$_id",
6     Receita_Total_Gerada: 1 } },
7   { $sort: { Categoria_Servico: 1 } }
8 ])
```

Algoritmo 11 – Consulta 4 (MongoDB)

{ Categoria_Servico: "Autenticação de Documento",	Receita_Total_Gerada: 45	}
{ Categoria_Servico: "Emissão de Certidão",	Receita_Total_Gerada: 120	}
{ Categoria_Servico: "Reconhecimento de Firma",	Receita_Total_Gerada: 75	}
{ Categoria_Servico: "Registro de Casamento",	Receita_Total_Gerada: 500	}
{ Categoria_Servico: "Registro de Imóvel",	Receita_Total_Gerada: 1000	}
{ Categoria_Servico: "Registro de Nascimento",	Receita_Total_Gerada: 240	}

Figura 4 – Resultado Consulta 4 (MongoDB)

- **Consulta 5** - Listar clientes com seus documentos de identificação

Recupera os dados de todos os clientes projetando o nome do titular, o tipo de documento e o número de registro, utilizando os documentos aninhados na coleção clientes.

```

1 db.clientes.find(
2   {},
3   { _id: 0, nome: 1,
4     "documentos.tipo_documento": 1,
5     "documentos.numero_documento": 1 }
6 ).limit(10)
```

Algoritmo 12 – Consulta 5 (MongoDB)

{ nome: "João Silva",	documentos: [{ tipo_documento: "RG",	numero_documento: "MG123456" }] }
{ nome: "Maria Oliveira",	documentos: [{ tipo_documento: "CPF",	numero_documento: "12345678902" }] }

```
{ nome: "Carlos Souza", documentos: [{ tipo_documento: "Certidão de Nascimento",
... }] }
{ nome: "Fernanda Lima", documentos: [{ tipo_documento: "RG", numero_documento:
"SP456789" }] }
{ nome: "Lucas Pereira", documentos: [{ tipo_documento: "CPF", numero_documento:
"12345678905"}] }
```

Figura 5 – Resultado Consulta 5 (MongoDB)

- **Consulta 6 - Funcionários ordenados por salário decrescente**

Seleciona todos os funcionários exibindo nome, cargo e remuneração atual, ordenando do maior para o menor salário.

```
1 db.funcionarios.find(
2   {},
3   { _id: 0, Colaborador: "$nome",
4     Cargo_Ocupado: "$cargo",
5     Remuneracao_Atual: "$salario" }
6 ).sort({ salario: -1 })
```

Algoritmo 13 – Consulta 6 (MongoDB)

```
{ Colaborador: "Roberta Lima", Cargo_Ocupado: "Tabeliã", Remuneracao_Atual:
7000 }
{ Colaborador: "Renato Alves", Cargo_Ocupado: "Supervisor", Remuneracao_Atual:
5000 }
{ Colaborador: "Amanda Costa", Cargo_Ocupado: "Escrevente", Remuneracao_Atual:
3600 }
{ Colaborador: "Paulo Henrique", Cargo_Ocupado: "Escrevente", Remuneracao_Atual:
3500 }
{ Colaborador: "Felipe Souza", Cargo_Ocupado: "Atendente", Remuneracao_Atual:
2800 }
```

Figura 6 – Resultado Consulta 6 (MongoDB)

- **Consulta 7 - Registros cartorários com o ato praticado em cada atendimento**

Exibe a natureza do registro, o livro, a folha e o ato cartorário praticado, utilizando os campos aninhados do documento de registro dentro de cada atendimento.

```

1 db.atendimentos.find(
2   {},
3   { _id: 0,
4     Natureza_Registro: "$registro.tipo_registro",
5     Livro_Cartorio:    "$registro.livro",
6     Folha_Cartorio:    "$registro.folha",
7     Ato_Praticado:     "$tipo_servico" }
8 ).limit(10)

```

Algoritmo 14 – Consulta 7 (MongoDB)

```

{ Natureza_Registro: "Firma",      Livro: "L01", Folha: "F12", Ato_Praticado:
"Reconhecimento de Firma" }
{ Natureza_Registro: "Documento", Livro: "L01", Folha: "F13", Ato_Praticado:
"Autenticação de Documento" }
{ Natureza_Registro: "Certidão",  Livro: "L02", Folha: "F20", Ato_Praticado:
"Emissão de Certidão" }
{ Natureza_Registro: "Nascimento", Livro: "L03", Folha: "F01", Ato_Praticado:
"Registro de Nascimento" }
{ Natureza_Registro: "Casamento", Livro: "L03", Folha: "F02", Ato_Praticado:
"Registro de Casamento" }

```

Figura 7 – Resultado Consulta 7 (MongoDB)

- **Consulta 8** - Clientes com RG emitido pelo estado de São Paulo

Filtra os clientes cujo campo RG inicia com o prefixo 'SP', identificando os clientes com documento de identidade emitido pelo estado de São Paulo.

```

1 db.clientes.find(
2   { rg: { $regex: "^SP" } },
3   { _id: 0,
4     Cliente_Paulista: "$nome",
5     Identidade_SP:    "$rg",
6     Endereco_Completo: "$endereco" }
7 )

```

Algoritmo 15 – Consulta 8 (MongoDB)

```

{ Cliente_Paulista: "Maria Oliveira",  Identidade_SP: "SP987654", Endereco: "Av.
Central, 200" }

```

```
{ Cliente_Paulista: "Fernanda Lima",  Identidade_SP: "SP456789", Endereco: "Rua Verde, 88" }
{ Cliente_Paulista: "Ana Costa",      Identidade_SP: "SP369258", Endereco: "Rua das Palmeiras, 50" }
{ Cliente_Paulista: "Patricia Gomes",  Identidade_SP: "SP753159", Endereco: "Av. Brasil, 300" }
{ Cliente_Paulista: "Bruno Martins",   Identidade_SP: "SP852456", Endereco: "Rua XV, 77" }
```

Figura 8 – Resultado Consulta 8 (MongoDB)

- **Consulta 9** - Ticket médio de atendimento por status

Agrupar os atendimentos pelo status (Concluído ou Pendente) e calcular o valor médio cobrado em cada grupo, evidenciando a diferença de ticket médio entre os dois status.

```
1 db.atendimentos.aggregate([
2   { $group: { _id: "$status_atendimento",
3     Ticket_Medio: { $avg: "$valor" } } },
4   { $project: { _id: 0,
5     Status_Atual: "$_id",
6     Ticket_Medio: { $round: ["$Ticket_Medio", 2] } } }
7 ])
```

Algoritmo 16 – Consulta 9 (MongoDB)

```
{ Status_Atual: "Concluído", Ticket_Medio: 81.67 }
{ Status_Atual: "Pendente", Ticket_Medio: 333.33 }
```

Figura 9 – Resultado Consulta 9 (MongoDB)

- **Consulta 10** - Painel completo de atendimentos com cliente e funcionário

Realiza dupla junção entre as coleções atendimentos, clientes e funcionarios, exibindo o número do protocolo, o cliente atendido, o funcionário responsável e o serviço executado.

```

1 db.atendimentos.aggregate([
2   { $lookup: { from: "clientes", localField: "id_cliente",
3     foreignField: "_id", as: "cliente" } },
4   { $lookup: { from: "funcionarios", localField: "id_funcionario",
5     foreignField: "_id", as: "funcionario" } },
6   { $unwind: "$cliente" },
7   { $unwind: "$funcionario" },
8   { $project: { _id: 0,
9     Num_Protocolo: "$_id",
10    Cliente_Atendido: "$cliente.nome",
11    Funcionario_Atendente: "$funcionario.nome",
12    Servico_Executado: "$tipo_servico" } }
13 ]).limit(5)

```

Algoritmo 17 – Consulta 10 (MongoDB)

```

{ Num_Protocolo: 1, Cliente: "João Silva",    Funcionario: "Paulo Henrique",
  Servico: "Reconhecimento de Firma"  }
{ Num_Protocolo: 2, Cliente: "Maria Oliveira",Funcionario: "Roberta Lima",
  Servico: "Autenticação de Documento" }
{ Num_Protocolo: 3, Cliente: "Carlos Souza",  Funcionario: "Felipe Souza",
  Servico: "Emissão de Certidão"      }
{ Num_Protocolo: 4, Cliente: "Fernanda Lima", Funcionario: "Amanda Costa",
  Servico: "Registro de Nascimento"    }
{ Num_Protocolo: 5, Cliente: "Lucas Pereira", Funcionario: "Renato Alves",
  Servico: "Registro de Casamento"    }

```

Figura 10 – Resultado Consulta 10 (MongoDB)

- **Consulta 11 - Documentos emitidos antes de 2018**

Desdobra os documentos aninhados dos clientes e filtra aqueles com data de emissão anterior a 01/01/2018, exibindo o tipo, número e data de emissão do documento.

```

1 db.clientes.aggregate([
2   { $unwind: "$documentos" },
3   { $match: {
4     "documentos.data_emissao": { $lt: new Date("2018-01-01") } } },
5   { $project: { _id: 0,
6     Tipo_Do_Documento: "$documentos.tipo_documento",
7     Num_Doc:             "$documentos.numero_documento",

```

```

8      Data_De_Emissao:   "$documentos.data_emissao" } }
9  ])

```

Algoritmo 18 – Consulta 11 (MongoDB)

```

{ Tipo_Do_Documento: "CPF",                      Num_Doc: "12345678902",
  Data_De_Emissao: 2017-08-15 }
{ Tipo_Do_Documento: "Certidão de Nascimento", Num_Doc: "CN1001",
  Data_De_Emissao: 2001-03-20 }
{ Tipo_Do_Documento: "CPF",                      Num_Doc: "12345678905",
  Data_De_Emissao: 2016-07-09 }
{ Tipo_Do_Documento: "CPF",                      Num_Doc: "12345678908",
  Data_De_Emissao: 2015-09-30 }
{ Tipo_Do_Documento: "Certidão de Nascimento", Num_Doc: "CN3003",
  Data_De_Emissao: 2000-01-01 }

```

Figura 11 – Resultado Consulta 11 (MongoDB)

- **Consulta 12 - Clientes com e-mail do domínio Gmail**

Utiliza expressão regular para filtrar clientes cujo endereço eletrônico termina com o domínio @gmail.com, exibindo nome completo e e-mail.

```

1 db.clientes.find(
2   { email: { $regex: "@gmail\.com$" } },
3   { _id: 0,
4     Nome_Completo:      "$nome",
5     Endereco_Eletronico: "$email" }
6 )

```

Algoritmo 19 – Consulta 12 (MongoDB)

```

{ Nome_Completo: "João Silva",      Endereco_Eletronico: "joao@gmail.com" }
{ Nome_Completo: "Maria Oliveira", Endereco_Eletronico: "maria@gmail.com" }
{ Nome_Completo: "Carlos Souza",   Endereco_Eletronico: "carlos@gmail.com" }
{ Nome_Completo: "Fernanda Lima",  Endereco_Eletronico: "fernanda@gmail.com" }
{ Nome_Completo: "Lucas Pereira",   Endereco_Eletronico: "lucas@gmail.com" }

```

Figura 12 – Resultado Consulta 12 (MongoDB)

- **Consulta 13** - Top 3 atendimentos de maior valor cobrado

Ordena os atendimentos pelo valor de forma decrescente e retorna apenas os três registros de maior custo, exibindo o serviço, o valor e a data do atendimento.

```
1 db.atendimentos.find(
2   {},
3   { _id: 0,
4     Servico_Premium: "$tipo_servico",
5     Custo_Maximo:    "$valor",
6     Data_Do_Registro: "$data_atendimento" }
7 ).sort({ valor: -1 }).limit(3)
```

Algoritmo 20 – Consulta 13 (MongoDB)

```
{ Servico_Premium: "Registro de Imóvel",    Custo_Maximo: 500, Data_Do_Registro:
2026-01-18 }
{ Servico_Premium: "Registro de Imóvel",    Custo_Maximo: 500, Data_Do_Registro:
2026-01-24 }
{ Servico_Premium: "Registro de Casamento", Custo_Maximo: 250, Data_Do_Registro:
2026-01-14 }
```

Figura 13 – Resultado Consulta 13 (MongoDB)

- **Consulta 14** - Quantidade de documentos processados por órgão emissor

Desdobra os documentos aninhados dos clientes e agrupa por órgão emissor, contando o volume total de documentos processados por cada entidade emissora.

```
1 db.clientes.aggregate([
2   { $unwind: "$documentos" },
3   { $group: { _id: "$documentos.orgao_emissor",
4     Volume_Processado: { $sum: 1 } } },
5   { $project: { _id: 0,
```

```

6   Orgao_Responsavel: "$_id",
7   Volume_Processado: 1 } },
8   { $sort: { Orgao_Responsavel: 1 } }
9 ])
```

Algoritmo 21 – Consulta 14 (MongoDB)

```

{ Orgao_Responsavel: "Cartório Central", Volume_Processado: 1 }
{ Orgao_Responsavel: "Cartório Norte",    Volume_Processado: 1 }
{ Orgao_Responsavel: "Cartório Sul",      Volume_Processado: 1 }
{ Orgao_Responsavel: "Receita Federal",   Volume_Processado: 6 }
{ Orgao_Responsavel: "SSP",               Volume_Processado: 6 }
```

Figura 14 – Resultado Consulta 14 (MongoDB)

- **Consulta 15** - Receita total arrecadada por dia de atendimento

Desdobra os documentos aninhados dos clientes e agrupa por órgão emissor, contando o volume total de documentos processados por cada entidade emissora.

```

1 db.atendimentos.aggregate([
2   { $group: {
3     _id: { $dateToString: {
4       format: "%Y-%m-%d",
5       date: "$data_atendimento" } } },
6     Total_Arrecadado_Dia: { $sum: "$valor" } } },
7   { $project: { _id: 0,
8     Data_Caixa: "$_id",
9     Total_Arrecadado_Dia: 1 } },
10  { $sort: { Data_Caixa: 1 } }
11 ])
```

Algoritmo 22 – Consulta 15 (MongoDB)

```

{ Data_Caixa: "2026-01-10", Total_Arrecadado_Dia: 25 }
{ Data_Caixa: "2026-01-11", Total_Arrecadado_Dia: 15 }
{ Data_Caixa: "2026-01-12", Total_Arrecadado_Dia: 40 }
{ Data_Caixa: "2026-01-13", Total_Arrecadado_Dia: 120 }
{ Data_Caixa: "2026-01-14", Total_Arrecadado_Dia: 250 }
```

Figura 15 – Resultado Consulta 15 (MongoDB)

- **Consulta 16** - Anos de empresa de cada funcionário

Calcula o tempo de serviço de cada funcionário em anos, comparando a data de admissão com a data atual, exibindo o colaborador e sua antiguidade na instituição.

```
1 db.funcionarios.aggregate([
2   { $project: { _id: 0,
3     Colaborador: "$nome",
4     Anos_Na_Empresa: { $dateDiff: {
5       startDate: "$data_admissao",
6       endDate:   "$$NOW",
7       unit:      "year" } } } }
8 ])
```

Algoritmo 23 – Consulta 16 (MongoDB)

```
{ Colaborador: "Paulo Henrique", Anos_Na_Empresa: 4 }
{ Colaborador: "Roberta Lima",   Anos_Na_Empresa: 6 }
{ Colaborador: "Felipe Souza",   Anos_Na_Empresa: 5 }
{ Colaborador: "Amanda Costa",   Anos_Na_Empresa: 3 }
{ Colaborador: "Renato Alves",   Anos_Na_Empresa: 7 }
```

Figura 16 – Resultado Consulta 16 (MongoDB)

- **Consulta 17** - Clientes que solicitaram Registro de Casamento

Calcula o tempo de serviço de cada funcionário em anos, comparando a data de admissão com a data atual, exibindo o colaborador e sua antiguidade na instituição.

```
1 db.atendimentos.aggregate([
2   { $match: { tipo_servico: "Registro de Casamento" } },
3   { $lookup: { from: "clientes", localField: "id_cliente",
```

```

4     foreignField: "_id", as: "cliente" } },
5   { $unwind: "$cliente" },
6   { $project: { _id: 0,
7     Nome_Dos_Noivos: "$cliente.nome",
8     Data_Da_Solicitacao: "$data_atendimento" } }
9 ] )

```

Algoritmo 24 – Consulta 17 (MongoDB)

```

{ Nome_Dos_Noivos: "Lucas Pereira", Data_Da_Solicitacao: 2026-01-14T15:45:00Z }
{ Nome_Dos_Noivos: "Mariana Lopes", Data_Da_Solicitacao: 2026-01-22T15:20:00Z }

```

Figura 17 – Resultado Consulta 17 (MongoDB)

- **Consulta 18** - Classificação dos serviços por faixa de preço

Aplica lógica condicional para classificar cada atendimento em 'Alto Custo' (valor $\geq$ R\$ 100,00) ou 'Baixo Custo' (valor $<$ R\$ 100,00), exibindo o serviço, o valor base e a faixa de preço.

```

1 db.atendimentos.aggregate([
2   { $project: { _id: 0,
3     Descricao_Servico: "$tipo_servico",
4     Valor_Base: "$valor",
5     Faixa_De_Preco: { $cond: {
6       if: { $gte: ["$valor", 100] },
7       then: "Alto Custo",
8       else: "Baixo Custo" } } } }
9 ] ).limit(6)

```

Algoritmo 25 – Consulta 18 (MongoDB)

```

{ Descricao_Servico: "Reconhecimento de Firma", Valor_Base: 25, Faixa_De_Preco: "Baixo Custo" }
{ Descricao_Servico: "Autenticação de Documento", Valor_Base: 15, Faixa_De_Preco: "Baixo Custo" }

```

```

{ Descricao_Servico: "Emissão de Certidão",      Valor_Base: 40,
  Faixa_De_Preco: "Baixo Custo" }
{ Descricao_Servico: "Registro de Nascimento",    Valor_Base: 120,
  Faixa_De_Preco: "Alto Custo" }
{ Descricao_Servico: "Registro de Casamento",     Valor_Base: 250,
  Faixa_De_Preco: "Alto Custo" }
{ Descricao_Servico: "Registro de Imóvel",         Valor_Base: 500,
  Faixa_De_Preco: "Alto Custo" }

```

Figura 18 – Resultado Consulta 18 (MongoDB)

- **Consulta 19** - Registros cartorários lançados no Livro L01

Filtra os atendimentos cujo registro cartorário aninhado pertence ao livro L01, exibindo o ato cartorário, o livro e a folha correspondente.

```

1 db.atendimentos.find(
2   { "registro.livro": "L01" },
3   { _id: 0,
4     Ato_Cartorario: "$tipo_servico",
5     Livro_De_Registro: "$registro.livro",
6     Folha_De_Registro: "$registro.folha" }
7 )

```

Algoritmo 26 – Consulta 19 (MongoDB)

```

{ Ato_Cartorario: "Reconhecimento de Firma",  Livro_De_Registro: "L01",
  Folha_De_Registro: "F12" }
{ Ato_Cartorario: "Autenticação de Documento", Livro_De_Registro: "L01",
  Folha_De_Registro: "F13" }
{ Ato_Cartorario: "Reconhecimento de Firma",  Livro_De_Registro: "L01",
  Folha_De_Registro: "F14" }
{ Ato_Cartorario: "Autenticação de Documento", Livro_De_Registro: "L01",
  Folha_De_Registro: "F15" }
{ Ato_Cartorario: "Reconhecimento de Firma",  Livro_De_Registro: "L01",
  Folha_De_Registro: "F16" }

```

Figura 19 – Resultado Consulta 19 (MongoDB)

- **Consulta 20** - Clientes com valor de atendimento acima da média geral

Utiliza subconsulta para calcular a média geral dos atendimentos e filtra apenas os clientes cujo atendimento teve valor superior a essa média, classificando-os como clientes VIP.

```

1 db.atendimentos.aggregate([
2   { $group: { _id: null, media: { $avg: "$valor" } } },
3   { $lookup: { from: "atendimentos",
4     pipeline: [], as: "todos" } },
5   { $unwind: "$todos" },
6   { $match: { $expr: { $gt: ["$todos.valor", "$media"] } } },
7   { $lookup: { from: "clientes",
8     localField: "todos.id_cliente",
9     foreignField: "_id", as: "cliente" } },
10  { $unwind: "$cliente" },
11  { $project: { _id: 0,
12    Cliente_VIP: "$cliente.nome",
13    Servico_Executado: "$todos.tipo_servico",
14    Valor_Gasto: "$todos.valor" } }
15 ])
```

Algoritmo 27 – Consulta 20 (MongoDB)

```

{ Cliente_VIP: "Lucas Pereira",  Servico_Executado: "Registro de Casamento",
Valor_Gasto: 250 }
{ Cliente_VIP: "Juliana Rocha",  Servico_Executado: "Registro de Imóvel",
Valor_Gasto: 500 }
{ Cliente_VIP: "Mariana Lopes",  Servico_Executado: "Registro de Casamento",
Valor_Gasto: 250 }
{ Cliente_VIP: "Aline Mendes",  Servico_Executado: "Registro de Imóvel",
Valor_Gasto: 500 }
```

Figura 20 – Resultado Consulta 20 (MongoDB)

- **Consulta 21** - Funcionários admitidos nos anos de 2020 ou 2021.

Filtra os funcionários cuja data de admissão pertence ao ano de 2020 ou 2021, utilizando o operador \$in combinado com a função \$year para extrair o ano do campo de data.

```

1 db.funcionarios.find(
2   { $expr: {
```

```

3     $in: [{ $year: "$data_admissao" }, [2020, 2021]] } },
4     { _id: 0,
5       Funcionario_Recente: "$nome",
6       Data_De_Entrada:      "$data_admissao" }
7 )

```

Algoritmo 28 – Consulta 21 (MongoDB)

```

{ Funcionario_Recente: "Roberta Lima",  Data_De_Entrada: 2020-03-15 }
{ Funcionario_Recente: "Felipe Souza",  Data_De_Entrada: 2021-07-20 }

```

Figura 21 – Resultado Consulta 21 (MongoDB)

- **Consulta 22 - Gasto acumulado por cliente nos atendimentos**

Agrupa os atendimentos por cliente, somando os valores gastos, e realiza junção com a coleção de clientes para exibir o nome do cliente e seu gasto total acumulado.

```

1 db.atendimentos.aggregate([
2   { $group: { _id: "$id_cliente",
3     total_gasto: { $sum: "$valor" } } },
4   { $lookup: { from: "clientes",
5     localField: "_id",
6     foreignField: "_id", as: "cliente" } },
7   { $unwind: "$cliente" },
8   { $project: { _id: 0,
9     Nome_Do_Cliente: "$cliente.nome",
10    Gasto_Acumulado: "$total_gasto" } },
11   { $sort: { Nome_Do_Cliente: 1 } }
12 ])

```

Algoritmo 29 – Consulta 22 (MongoDB)

```

{ Nome_Do_Cliente: "Aline Mendes",    Gasto_Acumulado: 500 }
{ Nome_Do_Cliente: "Ana Costa",       Gasto_Acumulado: 25  }
{ Nome_Do_Cliente: "Bruno Martins",   Gasto_Acumulado: 25  }
{ Nome_Do_Cliente: "Carlos Souza",    Gasto_Acumulado: 40  }
{ Nome_Do_Cliente: "Juliana Rocha",   Gasto_Acumulado: 500 }

```

Figura 22 – Resultado Consulta 22 (MongoDB)

- **Consulta 23** - Ficha de contato completa dos clientes

Utiliza o operador \$concat para montar uma string formatada com nome, CPF e telefone de cada cliente, gerando uma ficha de contato consolidada em campo único.

```
1 db.clientes.aggregate([
2   { $project: { _id: 0,
3     Ficha_De_Contato: { $concat: [
4       "$nome", " (CPF: ", "$cpf",
5       ") - Tel: ", "$telefone" ] } } }
6 ]).limit(5)
```

Algoritmo 30 – Consulta 23 (MongoDB)

```
{ Ficha_De_Contato: "João Silva (CPF: 12345678901) - Tel: (11)99999-1111" }
{ Ficha_De_Contato: "Maria Oliveira (CPF: 12345678902) - Tel: (11)99999-2222" }
{ Ficha_De_Contato: "Carlos Souza (CPF: 12345678903) - Tel: (11)99999-3333" }
{ Ficha_De_Contato: "Fernanda Lima (CPF: 12345678904) - Tel: (11)99999-4444" }
{ Ficha_De_Contato: "Lucas Pereira (CPF: 12345678905) - Tel: (11)99999-5555" }
```

Figura 23 – Resultado Consulta 23 (MongoDB)

- **Consulta 24** - Base para comissão por funcionário em atendimentos concluídos

Filtra apenas os atendimentos com status 'Concluído', agrupa por funcionário e soma os valores dos atendimentos realizados, gerando a base de cálculo para comissões.

```
1 db.atendimentos.aggregate([
2   { $match: { status_atendimento: "Concluído" } },
3   { $group: { _id: "$id_funcionario",
4     Base_Para_Comissao: { $sum: "$valor" } } },
5   { $lookup: { from: "funcionarios",
6     localField: "_id",
7     foreignField: "_id", as: "funcionario" } },
```

```

8   { $unwind: "$funcionario" },
9   { $project: { _id: 0,
10     Nome_Do_Colaborador: "$funcionario.nome",
11     Base_Para_Comissao: 1 } },
12   { $sort: { Nome_Do_Colaborador: 1 } }
13 ])
```

Algoritmo 31 – Consulta 24 (MongoDB)

```

{ Nome_Do_Colaborador: "Amanda Costa",   Base_Para_Comissao: 160 }
{ Nome_Do_Colaborador: "Felipe Souza",   Base_Para_Comissao: 80  }
{ Nome_Do_Colaborador: "Paulo Henrique", Base_Para_Comissao: 65  }
{ Nome_Do_Colaborador: "Renato Alves",   Base_Para_Comissao: 525 }
{ Nome_Do_Colaborador: "Roberta Lima",   Base_Para_Comissao: 150 }
```

Figura 24 – Resultado Consulta 24 (MongoDB)

- **Consulta 25** - Último atendimento realizado no cartório

Ordena os atendimentos pela data de forma decrescente e retorna apenas o registro mais recente, identificando o último cliente atendido e a data do atendimento.

```

1 db.atendimentos.aggregate([
2   { $sort: { data_atendimento: -1 } },
3   { $limit: 1 },
4   { $lookup: { from: "clientes",
5     localField: "id_cliente",
6     foreignField: "_id", as: "cliente" } },
7   { $unwind: "$cliente" },
8   { $project: { _id: 0,
9     Ultimo_Cliente_Atendido: "$cliente.nome",
10    Data_Mais_Recente: "$data_atendimento" } }
11 ])
```

Algoritmo 32 – Consulta 25 (MongoDB)

```
{ Ultimo_Cliente_Atendido: "Aline Mendes", Data_Mais_Recente: 2026-01-24T13:50:00Z
}
```

Figura 25 – Resultado Consulta 25 (MongoDB)

- **Consulta 26 - Ranking de funcionários por salário**

Ordena os funcionários pelo salário de forma decrescente e atribui uma posição de ranking a cada colaborador, utilizando o operador \$push com includeArrayIndex para numeração sequencial.

```
1 db.funcionarios.aggregate([
2   { $sort: { salario: -1 } },
3   { $group: { _id: null,
4     funcionarios: { $push: "$$ROOT" } } },
5   { $unwind: {
6     path: "$funcionarios",
7     includeArrayIndex: "posicao" } },
8   { $project: { _id: 0,
9     Posicao_No_Ranking: { $add: ["$posicao", 1] },
10    Empregado: "$funcionarios.nome",
11    Salario_Atual: "$funcionarios.salario" } }
12 ])
```

Algoritmo 33 – Consulta 26 (MongoDB)

```
{ Posicao_No_Ranking: 1, Empregado: "Roberta Lima", Salario_Atual: 7000 }
{ Posicao_No_Ranking: 2, Empregado: "Renato Alves", Salario_Atual: 5000 }
{ Posicao_No_Ranking: 3, Empregado: "Amanda Costa", Salario_Atual: 3600 }
{ Posicao_No_Ranking: 4, Empregado: "Paulo Henrique", Salario_Atual: 3500 }
{ Posicao_No_Ranking: 5, Empregado: "Felipe Souza", Salario_Atual: 2800 }
```

Figura 26 – Resultado Consulta 26 (MongoDB)

- **Consulta 27** - Quantidade de registros cartorários por tipo

Agrupar os atendimentos pelo tipo do registro aninhado e conta a quantidade de entradas de cada classe de registro, gerando um relatório de volume cartorário por natureza.

```
1 db.atendimentos.aggregate([
2   { $group: { _id: "$registro.tipo_registro",
3     Qtd_De_Entradas: { $sum: 1 } } },
4   { $project: { _id: 0,
5     Classe_De_Registro: "$_id",
6     Qtd_De_Entradas: 1 } },
7   { $sort: { Classe_De_Registro: 1 } }
8 ])
```

Algoritmo 34 – Consulta 27 (MongoDB)

```
{ Classe_De_Registro: "Casamento",  Qtd_De_Entradas: 2 }
{ Classe_De_Registro: "Certidão",    Qtd_De_Entradas: 3 }
{ Classe_De_Registro: "Documento",   Qtd_De_Entradas: 3 }
{ Classe_De_Registro: "Firma",       Qtd_De_Entradas: 3 }
{ Classe_De_Registro: "Imóvel",      Qtd_De_Entradas: 2 }
{ Classe_De_Registro: "Nascimento",  Qtd_De_Entradas: 2 }
```

Figura 27 – Reajuste salarial de 10% para os Escreventes

- **Consulta 28** - Reajuste salarial de 10% para os Escreventes:

Atualiza o salário de todos os funcionários com cargo 'Escrevente', aplicando um reajuste de 10% por meio do operador \$mul, e exibe os valores antes e depois da atualização.

```
1 db.funcionarios.updateMany(
2   { cargo: "Escrevente" },
3   { $mul: { salario: 1.10 } }
4 )
5
6 // Verificação após atualização:
```

```

7 db.funcionarios.find(
8   { cargo: "Escreveinte" },
9   { _id: 0, nome: 1, cargo: 1, salario: 1 }
10 )

```

Algoritmo 35 – Consulta 28 (MongoDB)

```

// Resultado da operação:
{ acknowledged: true, matchedCount: 2, modifiedCount: 2 }

// Valores atualizados:
{ nome: "Paulo Henrique", cargo: "Escreveinte", salario: 3850 }
{ nome: "Amanda Costa", cargo: "Escreveinte", salario: 3960 }

```

Figura 28 – Resultado Consulta 28 (MongoDB)

- **Consulta 29** - Clientes com atendimento de valor igual ou superior a R\$ 250,00

Utiliza o operador \$exists em combinação com \$gte para identificar clientes que possuem ao menos um atendimento de alto valor, exibindo nome, CPF e endereço residencial.

```

1 db.atendimentos.aggregate([
2   { $match: { valor: { $gte: 250 } } },
3   { $lookup: { from: "clientes",
4     localField: "id_cliente",
5     foreignField: "_id", as: "cliente" } },
6   { $unwind: "$cliente" },
7   { $project: { _id: 0,
8     Cliente_Alto_Valor: "$cliente.nome",
9     CPF_Do_Cliente: "$cliente.cpf",
10    Endereco_Residencial: "$cliente.endereco" } }
11 ])

```

Algoritmo 36 – Consulta 29 (MongoDB)

```

{ Cliente_Alto_Valor: "Lucas Pereira", CPF: "12345678905", Endereco: "Av.
Paulista, 900" }
{ Cliente_Alto_Valor: "Juliana Rocha", CPF: "12345678909", Endereco: "Rua da Paz,
400" }

```

```
{ Cliente_Alto_Valor: "Mariana Lopes", CPF: "12345678913", Endereco: "Av.
Independência, 450" }
{ Cliente_Alto_Valor: "Aline Mendes", CPF: "12345678915", Endereco: "Rua
Primavera, 145" }
```

Figura 29 – Resultado Consulta 29 (MongoDB)

- **Consulta 30** - Painel gerencial de desempenho por funcionário

Gera um painel executivo agrupando os atendimentos por funcionário, calculando a quantidade de atendimentos concluídos, a quantidade de pendentes e o volume financeiro total gerado, ordenando pelo maior volume.

```
1 db.atendimentos.aggregate([
2   { $group: { _id: "$id_funcionario",
3     Qtd_Concluidos: { $sum: { $cond: [
4       { $eq: ["$status_atendimento", "Concluído"] }, 1, 0 ] } },
5     Qtd_Pendentes: { $sum: { $cond: [
6       { $eq: ["$status_atendimento", "Pendente"] }, 1, 0 ] } },
7     Volume_Financeiro_Total: { $sum: "$valor" } } },
8   { $lookup: { from: "funcionarios",
9     localField: "_id",
10    foreignField: "_id", as: "funcionario" } },
11   { $unwind: "$funcionario" },
12   { $project: { _id: 0,
13     Colaborador_Responsavel: "$funcionario.nome",
14     Qtd_Concluidos: 1, Qtd_Pendentes: 1,
15     Volume_Financeiro_Total: 1 } },
16   { $sort: { Volume_Financeiro_Total: -1 } }
17 ])
```

Algoritmo 37 – Consulta 30 (MongoDB)

```
{ Colaborador: "Renato Alves", Qtd_Concluidos: 2, Qtd_Pendentes: 1,
Volume_Total: 775 }
{ Colaborador: "Amanda Costa", Qtd_Concluidos: 2, Qtd_Pendentes: 1,
Volume_Total: 660 }
{ Colaborador: "Felipe Souza", Qtd_Concluidos: 2, Qtd_Pendentes: 1,
Volume_Total: 330 }
{ Colaborador: "Roberta Lima", Qtd_Concluidos: 3, Qtd_Pendentes: 0,
Volume_Total: 150 }
{ Colaborador: "Paulo Henrique", Qtd_Concluidos: 3, Qtd_Pendentes: 0,
Volume_Total: 65 }
```

Figura 30 – Resultado Consulta 30 (MongoDB)

4 CONCLUSÃO

O desenvolvimento deste trabalho possibilitou a aplicação prática dos conceitos relacionados aos bancos de dados não relacionais, por meio da adaptação do projeto relacional do Sistema de Gestão Cartorária para um banco de dados NoSQL utilizando o MongoDB. Essa etapa permitiu compreender as principais diferenças entre os modelos relacional e não relacional, evidenciando as características do modelo orientado a documentos e sua capacidade de armazenar e organizar informações de forma flexível, sem comprometer as regras de negócio definidas para o sistema.

A utilização do MongoDB demonstrou que a organização dos dados em coleções e documentos representa uma alternativa eficiente para aplicações que necessitam de maior flexibilidade e facilidade de evolução da estrutura de dados. Além disso, a elaboração e execução das trinta consultas permitiram validar a modelagem desenvolvida, comprovando que a base de dados atende aos requisitos estabelecidos para o gerenciamento das informações do sistema cartorário.

No que se refere ao desenvolvimento do projeto, todas as etapas previstas foram concluídas conforme o planejamento proposto. Partindo do banco de dados relacional desenvolvido anteriormente, realizou-se o estudo dos conceitos de bancos de dados NoSQL, a adaptação da modelagem para o MongoDB, a implementação das coleções, a inserção dos dados e a validação da estrutura por meio das consultas realizadas. Dessa forma, os objetivos estabelecidos para o trabalho foram plenamente alcançados.

Como sugestão para trabalhos futuros, propõe-se a ampliação da base de dados por meio da inclusão de novos módulos voltados ao ambiente cartorário, como um módulo de agendamento eletrônico de atendimentos, controle de fluxo e autenticação digital de documentos em lote (procurações, escrituras e certidões), além da implementação de um módulo financeiro para gerenciamento de emolumentos, taxas extrajudiciais, faturamento, emissão de notas fiscais e controle de repasses institucionais."

REFERÊNCIAS

BANKER, Kyle. *MongoDB em ação*. São Paulo: Novatec Editora, 2013.

MICROSOFT. *Microsoft SQL Server*. Disponível em:
<https://www.microsoft.com/sql-server>. Acesso em: 26 maio 2026.

MONGODB INC. *MongoDB Documentation*. Disponível em:
<https://www.mongodb.com/docs>. Acesso em: 26 maio 2026.

NIELD, Thomas. *Introdução à linguagem SQL*. São Paulo: Novatec Editora, 2016.

ROB, Peter; CORONEL, Carlos. *Sistemas de banco de dados: projeto, implementação e administração*. São Paulo: Cengage Learning, 2011.

SADALAGE, Pramod J.; FOWLER, Martin. *NoSQL essencial: um guia rápido para o mundo emergente da persistência poliglota*. São Paulo: Novatec Editora, 2013.